

Part 3 — Proof of concept, blockers, schedule and release

Documentation for the brief's Part 3: "how to proceed with engineering to Proof of Concept", "the list of potential blockers", "what are known and trivial and what are the estimates arrival time", "implementation schedules", and "how to have a successful and highly quality release". The three outputs the brief asks for map to sections as follows: how to proceed to next steps → §3 and §7; how to evaluate the proof of concept → §1 and §5; release plan and time estimations → §4 and §6.

Numbers are labelled **measured**, **assumed** or **derived** as in Part 2.

1. PoC scope and success criteria

The PoC runs the Part 1 crawler through the Part 2 pipeline (ingest → frontier → SQS → worker fleet → S3 + ClickHouse → read API) in one AWS region, and answers the assumptions the design rests on. It is two runs, because the two input regimes have different binding constraints (Part 2 §2):

Run	Input	What it measures	Why this shape
A — few-domain	the three hosts the brief names; up to 1M URLs sampled from their sitemaps or a customer list; two crawl cycles a week apart	accepted rate per host (the assumed 10 requests a second per host); block rate and robots_state per host; page-size distribution (page size); unchanged rate on recrawl (the unchanged-on-recrawl rate); JS-only share (Part 2 Appendix A decision 13)	the brief's own example; at 10 req/s per host 1M URLs take 9.3 h (derived: $1e6 / 30 / 3,600$) — this run cannot test fleet throughput and is not asked to
B — many-domain	25M URLs across $\geq 10,000$ hosts (the customer's long tail, or hosts whose robots.txt permits crawling), one cycle; plus two synthetic loads: a frontier table filled with $1e9$ generated rows over 1M hosts, and a ClickHouse inserter at 3,858 rows/s	sustained fetch rate, parse CPU on target cores (parse cost per page), cost per million (cost per million), ClickHouse insert behaviour (Part 2 Appendix A decision 7), point-lookup p99 (decision 10)	throughput can only be measured where politeness is not the ceiling

Out of scope for the PoC: multi-region, JavaScript rendering, the full recrawl scheduler beyond the priority formula, and any read API beyond point lookup and the month/domain scan.

Success criteria. The PoC passes when every row of §5 has a measured value and each pass/fail row passes, and when the Part 2 cost model, re-run with the measured page size, parse cost and unchanged rate, lands within 30% of the pre-PoC figure or the difference is explained line by line.

2. Blockers

Each blocker is classed **known / trivial** (the method is known; only effort is needed) or **risky / unknown** (the outcome is uncertain; measurement or a decision outside engineering resolves it). "Arrival time" is when it is expected to be resolved on the §3 schedule; "unblocked by" names what resolves it.

#	Blocker	Class	Arrival time	Unblocked by
B1	Ingesting an 800 GB URL file (the 800 GB list file)	known / trivial	week 2	byte-range parallel read from S3; external sort by url_hash; 3 engineer-days (assumed)

#	Blocker	Class	Arrival time	Unblocked by
B2	Reading the source list from MySQL	known / trivial	week 2	replica + keyset pagination; 1 day
B3	Malformed input (invalid UTF-8, null bytes, stray CR)	known / trivial	week 2	sanitise at ingest; reject and count, never drop silently; 1 day
B4	Frontier scheduler: refill, back queues, token buckets (Part 2 §4.2)	known / trivial	week 3	frontier table + Redis lists and Lua token bucket + selector loop; 5 days
B5	Raw-object batching with offset pointers (decision 6)	known / trivial	week 3	buffer + flush + ranged-GET reader; 2 days
B6	Fleet deployment and autoscaling	known / trivial	week 3	ECS on Spot ASG; scaling rule from Part 2 §4.4; 4 days
B7	Monitoring wiring (Part 2 §7)	known / trivial	week 5	metrics already defined; dashboards and alarms; 4 days
B8	Page-size distribution (page size)	risky / unknown	week 1	Run A sample of 10,000 pages; a 5x swing changes storage and bandwidth lines
B9	Parse CPU on target cores (parse cost per page)	risky / unknown	week 1	profile on c7g; replaces the 1.5x assumption; scales the largest cost line
B10	Unchanged rate on recrawl (the unchanged-on-recrawl rate)	risky / unknown	week 5	Run A cycle 2; the largest saving in the model
B11	ClickHouse insert batching and merge lag at target rate	risky / unknown	week 5	Run B; decides whether ClickHouse or a managed alternative ships
B12	Point-lookup p99 under concurrent inserts (decision 10)	risky / unknown	week 6	load test; decides whether a KV summary tier is added
B13	Anti-bot blocking on target hosts, including blocks disguised as maintenance or error pages (no_content)	risky / unknown	week 1 (measured), decision by week 2	measured per host in Run A with a browser-User-Agent probe on every no_content and blocked host; the policy question (accept the coverage ceiling, or negotiate access) is the customer's and is raised in week 1
B14	Per-host accepted rate (the assumed 10 requests a second per host)	risky / unknown	week 1 (measured); negotiation open-ended	1 / 5 / 10 req/s probes per host; anything above is a commercial agreement, not engineering
B15	JavaScript-only share on target hosts (Part 2 Appendix A decision 13)	risky / unknown	week 1	detector on the Run A sample; > 10% triggers a separately funded rendering workstream
B16	Egress IP reputation (decision 11)	risky / unknown	week 4	block rate by egress IP during Run B; reverses to NAT with Elastic IPs if a host requires an allow-list
B17	Classification and topic quality	risky / unknown	week 5	the labelled set in §5; unmeasured until it exists
B18	Legal and terms-of-service position: crawling hosts that disallow it, and probing hosts with a browser User-Agent	risky / unknown	entry gate for phase 0 ; owner: customer counsel or ours, named before day 1	a written position before any browser-User-Agent probe; engineering enforces robots_state either way
B19	Frontier refill latency on a 1.2 TB table at 1M hosts (Part 2 Appendix A, decision 3)	risky / unknown	week 4	Run B; if top-k refill cannot sustain the release rate, back-queue heads are pre-materialised by a batch job
B20	AWS Spot vCPU quota (Run B needs ~800 vCPUs; production ~4,000)	known / trivial, external	filed day 1; days to weeks	quota increase request; run smaller until granted

#	Blocker	Class	Arrival time	Unblocked by
B21	Customer MySQL access: replica, network path, credentials	known / trivial, external	week 1	customer provisions; ingest from file until then
B22	PoC infrastructure budget (~\$15–20k over ten weeks at list prices)	known / trivial, external	before phase 1	approval by the budget owner named in the phase-0 gate
B23	Security review of a fleet with public egress IPs	known / trivial, external	before phase 2	review request filed in week 1

The two blockers that decide the project's shape are B13 and B14: both are measured in week 1, and both change what can be promised rather than how it is built.

3. Implementation schedule

Ten calendar weeks to a single-region system ready for a staged release. Each phase de-risks something named and ends at a gate that must pass before the next begins.

Phase	Weeks	Delivers	De-risks	Exit gate
0 — Measure	1	Run A sample of 10,000 pages on the three hosts: page sizes, block rate, robots_state, JS-only share, rate probes; parse CPU on c7g; cost model re-run	B8, B9, B13, B14, B15	measured values for page size, parse cost, accepted rate per host; cost model revised and re-approved if any line moved > 50%; the anti-bot policy question raised with its owner
1 — Pipeline	2–3	ingest (file + MySQL), frontier table and back queues, SQS, worker fleet, raw-object writer, ClickHouse schema; single region, fixed fleet size	B1–B6; that the components fit together	10,000 URLs end to end; for every URL the page_fetch row and a Part 1 CLI run within the same hour agree on status, reason, robots_state, http_status, title and page_type (body and word count are compared only for hosts that do not serve per-fetch variants; Amazon does — README) ; a killed worker loses no URL
2 — Scale runs	4–6	Run A full (1M URLs, two cycles); Run B (25M URLs, ≥ 10,000 hosts, plus the synthetic frontier and insert loads); ClickHouse insert tuning; monitoring wired	B7, B10, B11, B16, B19	\$5 throughput, cost and coverage rows measured; cost per million within 30% of the revised model or explained
3 — Serving	7	read API, Redis cache, CDN; point lookup and month/domain scan; load test	B12	p95 < 200 ms cached and < 1 s on miss at 250 req/s for 1 h with inserts running; p99 on miss recorded against the 100 ms KV threshold
4 — Hardening	8–10	autoscaling, failure injection, runbooks, alerting review, canary path, on-call rota	operability by people who did not build it	\$5 operations rows pass; runbooks executed by a non-author; rollback rehearsed

Phase 2 has three weeks because Run A needs two crawl cycles a week apart and Run B must follow them; phase 4 has three because failure injection produces findings that need fixing. The estimates in §4 sum to 20 engineer-weeks likely, which is the two engineers at full allocation for the ten weeks; there is no hidden slack beyond that, and the high case is 15 calendar weeks.

4. Estimates

Team shape assumed: **two backend engineers**, full time, with a cloud account provisioned and a customer contact who can answer the anti-bot and rate-negotiation questions in week 1. No JavaScript rendering, no multi-region, no customer-facing UI.

Phase	Low	Likely	High	What moves it
0 — Measure	1.5 eng-wk	2	2.5	host probes need repeating if blocked early
1 — Pipeline	3	4	5	frontier correctness (refill, release, re-release after a Redis loss)
2 — Scale runs	4	6	9	two Run A cycles a week apart, Run B, synthetic loads, insert tuning, the labelled sample (~1 eng-wk)
3 — Serving	1.5	2	3.5	p99 miss latency forcing the KV tier (adds ~1 week)
4 — Hardening	4	6	10	failure-injection findings; a replay run for the failure tests
Total	14 eng-wk	20	30	

Derived: 20 engineer-weeks over 10 calendar weeks is the two-engineer team at full allocation; the high case is 15 calendar weeks. A third engineer with ClickHouse experience from week 3 would bring the likely case back to 8 calendar weeks. The estimates are engineering effort only; the rate negotiation in B14 and the legal position in B18 are outside them and can run in parallel or gate scale-up regardless.

Assumptions that, if false, move the whole table: (a) the Part 1 crawler's extraction quality is acceptable as-is on the target hosts (else parser work enters phase 2); (b) the customer's list arrives in the brief's form (file or MySQL) without a new connector; (c) no host requires authenticated or rendered access for its core pages.

5. Evaluating the proof of concept

A PoC that produces no numbers has proven nothing. Every row has a method and a pass/fail rule.

5.1 Extraction and classification quality

Method: a **labelled sample of 500 pages** stratified across the three hosts and the Run B long tail, labelled by two reviewers for title, `page_type`, body cleanliness and topic relevance. 500 is chosen so that a 95% target is estimated to ± 2 points at 95% confidence (derived: $1.96 \times \sqrt{(0.95 \times 0.05 / 500)} = 1.9$ points); doubling the sample halves nothing useful at this stage.

Metric	Pass	Method
Title correct	$\geq 98\%$	reviewer agreement with title
<code>page_type</code> correct	$\geq 95\%$	reviewer label vs <code>page_type</code> ; confusion matrix reported
Body free of navigation, ads, footer	$\geq 90\%$	reviewer judgement on the first 500 characters and the last 200
Topics: at least 3 of the top 5 judged relevant by both reviewers	$\geq 70\%$	two reviewers, disagreements adjudicated
Invented metadata (author or date absent from the page)	0	grep of the value in the raw HTML

Topic relevance carries the loosest threshold because the extractor is unsupervised (Part 1 README); `page_type` carries a tight one because it reads declared markup. The labelled set is kept and versioned: every later parser change is measured against it.

5.2 Throughput and cost (Run B)

Metric	Pass	Method
Sustained fetch rate	$\geq 1,000$ URLs/s for 6 h on a fleet of ≤ 70 instances ($1,000 \times 0.75$ s / 16 ≈ 47 at full utilisation; 67 at the 70% the design runs)	fleet metrics; 25M URLs is about 7 h at that rate
Parse CPU per page on c7g	measured; replaces the parse-cost assumption	in-fleet profiling, $n \geq 10,000$
Unit costs per line (instance-hours, PUTs, bytes stored)	measured and written back into the ledger; no percentage gate at PoC scale, where Spot price noise exceeds it	Cost Explorer on tagged resources
ClickHouse parts per partition; insert lag	< 300 parts; lag < 5 min at 3,858 rows/s from the synthetic inserter	system tables

5.3 Coverage (Run A)

Metric	Pass	Method
Accepted rate per host	measured at 1 / 5 / 10 req/s; no target	429 and 403 rate per probe level
Block rate per host	measured, no target	blocked / fetched, by reason; plus <code>no_content</code> hosts whose browser-User-Agent probe succeeds
Error rate per host	$< 0.5\%$ excluding blocked	error / fetched, by reason
Unchanged rate on cycle 2, per host	measured; replaces the 60% upper bound in the cost model	raw-body <code>content_hash</code> equality; hosts serving per-fetch variants are reported separately
JS-only share per host	measured; $> 10\%$ triggers Part 2 Appendix A decision 13's rendering workstream	detector on the sample
Page-size distribution	measured p50 / p90 / p99; replaces the page-size assumption	response sizes

Block rate has no target on purpose: a target creates pressure to defeat bot detection. It is reported.

5.4 Operations (phase 4)

Metric	Pass	Method
Worker loss	zero URLs lost, zero duplicates in <code>page_current</code>	kill 20% of workers mid-run; reconcile counts
Redis loss	≤ 60 s of URLs re-released; none lost	fail over Redis during Run B
Queue drain after a 2 h pause	< 2 h	pause consumers, resume, time to empty
Alert precision	every alert in a one-week soak is actionable	review each alert against Part 2 §7 actions
Runbook usability	each runbook completed by a non-author without help	dry run

6. Release plan

6.1 Staged rollout

Stage	Volume	Duration	Proceed when
Canary	$\leq 10\%$ of every host's budget, routed to the new fleet by a weight on the selector (Part 2 §4.2)	48 h	error rate $< 1\%$, parse failures $< 5\%$, block rate per host within PoC bounds; no paging alert
Limited	10%, all hosts in the batch	1 week	cost per million within 20% of model; SLOs met
Broad	50%	1 week	SLOs met; no manual intervention
Full	100%	—	error budget on plan

Each stage runs beside the previous fleet; the selector weights each host's releases between the two fleets, so rollback is setting the new fleet's weight to zero, not a redeploy.

6.2 Rollback triggers (automatic)

- `error` rate above 1% for 15 minutes on a first release, or above 2× the 7-day baseline once one exists
- parse failure rate above 5%
- `page_current` rows missing required fields above 0.1%
- hourly cost proxy (instance-hours × Spot price, per URL fetched) above 150% of model for 6 hours; Cost Explorer lags a day and cannot drive an automatic trigger

Idempotency makes rollback cheap: a re-run of any URL produces the same rows.

6.3 Quality gates on every change

- `make check` green (lint, strict types, tests) on every commit
- labelled-set regression (§5.1) on every parser or classifier change; a drop > 1 point blocks merge
- the ledger script re-run on every change to a capacity or cost input; Part 2 tables regenerated from it
- canary before any fleet-wide deploy

6.4 Runbooks, written before launch

Queue backlog growth; a host starting to block; ClickHouse insert lag; parse-failure spike after a site redesign; spot capacity shortfall; Redis failover; rollback. Each runbook is executed once by a non-author in phase 4.

6.5 On-call

One primary, one secondary, weekly rotation across the two engineers plus one additional engineer trained in phase 4 so that a two-person team is not on call every week. Paging alerts are the Part 2 §7 rows marked as actions on the fleet or a store; per-host coverage changes are tickets, not pages. Every page has a runbook link.

6.6 What "high quality" means for this system

1. **Correct over complete.** A missing field is acceptable; an invented one is not (the Part 1 extractor returns `null` rather than a guessed author or date).
2. **Honest coverage.** Every batch report states fetched, blocked (by reason), errored (by reason), robots-disallowed and JS-only counts per host.
3. **Operable by others.** Runbooks, dashboards and alerts are tested by people who did not build the system before it takes real load.

7. Next steps beyond the PoC

Ranked by how much the design changes if the finding is adverse (mirrors Part 2 §9):

1. Settle the per-host rate and the anti-bot policy with the customer (B13, B14); everything promised in the few-domain regime follows from these.
2. Re-run the cost model with measured page size, parse cost and unchanged rate and re-approve the budget.
3. Decide the JS-rendering workstream from the measured share (B15).
4. Decide the KV summary tier from the measured point-lookup p99 (B12).
5. Run the fair columnar-vs-row comparison on target hardware (Part 2 §9 item 8) before quoting any multiple.
6. Second region and cross-region read replica, only after the single-region SLOs have held for a full monthly batch.